Implementing and Evaluating an RLHF Pipeline for Open-Source LLMs

Jaden Machado
Advanced AIML
A1992876@adelaide.au.edu

March 19, 2026

Abstract

Incorporating complex human preferences with large language models (LLMs) is a very difficult task. Fortunately, Reinforcement Learning with Human feed-back (RLHF) exists as a powerful technique to assist with this. It helps models produce outputs that are more accurate and harmless. My report presents a detailed, hands-on implementation of an RLHF pipeline using open-source LLMs and datasets. I trained the SFT models on cleaned yahma/alpaca [Vanroy, 2023, vinhtran2611, 2023], used SST-2 [Moura, 2025, Data, 2025] for prompts during RL, and built a tiny preference dataset (modeled after Anthropic HH-RLHF [Bai et al., 2022]) for reward modeling. What I understood was **GPT-2** with 124 million parameters responded well to RLHF, but **DistilGPT2** with 82 million parameters didn't. Its compressed architecture and higher perplexity [Li et al., 2021] just couldn't keep up with more complex generation tasks. This experience taught me a lot about the power of RLHF and the entire pipeline process.

1 Introduction

The rapid advancement of Large Language Models (LLMs) has led to models with impressive generative capabilities. However, the objective of standard language modeling—predicting the next token in a vast corpus of internet text—does not inherently align with the diverse, nuanced, and often subjective intent of a human user. In practice, this means models might make stuff up, say biased things, or just give completely useless answers.

RLHF is basically a way to train models so they better match what people are looking for. Instead of using a fixed metric like accuracy, RLHF learns from comparisons—showing the model which responses people prefer and which they don't. Then, during RL fine-tuning, the model learns to maximize that reward signal.

This report walks through how I built an RLHF pipeline step by step. My main goal was to get hands-on with every stage—data prep, training, evaluation—and also compare how two different models, **GPT-2** and **DistilGPT2**, respond to RLHF. Since DistilGPT2 is a compressed version of GPT-2 [Li et al., 2021], comparing them was a good way to see how much capacity matters for RLHF. Section 2 goes over the key papers I built on. In Section 3, I explain my

methodology—what data I used, which models, and how I ran each stage. Section 4 has the results, and Section 5 covers limitations and what I'd try next time.

Research Question

Does model capacity significantly impact the effectiveness of Reinforcement Learning from Human Feedback (RLHF) in small, open-source language models?

2 Literature Review

AI alignment isn't a new problem—people have been trying to get models to do what humans want for years. For LLMs, the real breakthrough came with InstructGPT Ouyang et al. [2022]. What Ouyang and his team showed was pretty surprising: a 1.3B parameter model fine-tuned with human feedback was actually preferred over the massive 175B-parameter original. That was a huge moment—it proved that how you align a model matters just as much as how big it is.

The InstructGPT pipeline is based on this three-step pipeline:

1. **Supervised Fine-Tuning (SFT):** First, I fine-tuned the base model on human-written examples to teach it the right format and style. I used the yahma/alpaca-cleaned dataset for this. [Vanroy, 2023, vinhtran2611, 2023].

2. **Reward Modeling (RM):** So the idea here is simple: I needed a model that could act like a judge. I trained it on pairs of responses—one that humans liked and one they didn't—and it learned to score the good one higher. Datasets like Anthropic's HH-RLHF [Bai et al., 2022] are perfect for this because they already have all those human preferences labeled.

3. **PPO Fine-Tuning:** For the final stage, I used PPO. The setup was a loop: model generates text → reward model scores it → PPO updates the model. Over and over. The goal was simple—get the model to consistently produce responses that the reward model would give high scores to.

Since InstructGPT came out, people have come up with other approaches too. DPO [Rafailov et al., 2023] is a cool alternative—it skips the reward model and RL entirely and just uses a classification loss. But for this project, I wanted to really understand the original pipeline first. With my compute limits, it made more sense to get the basics right before experimenting with fancier methods.

3 Methodology

The project was executed as a combination of four Jupyter notebooks. They mirror the three stages of the RLHF pipeline. The code was run using a single NVIDIA T4 GPU provided by Google Colab. The complete workflow is illustrated in Figure 1.

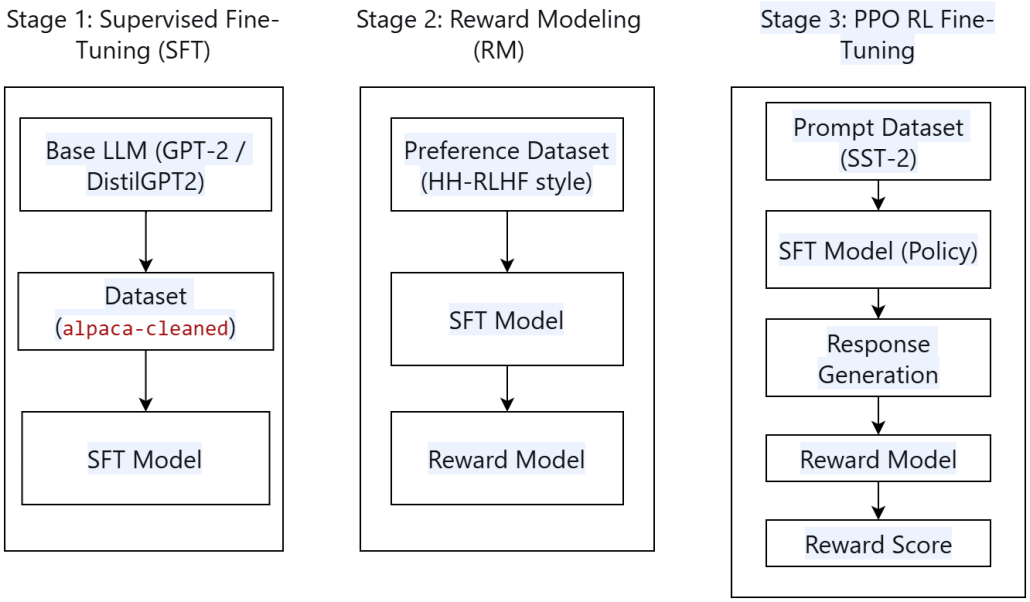


Figure 1: The three-stage RLHF pipeline implemented in this project. 1. Supervised Fine-Tuning (SFT) on instruction data. 2. Reward Model (RM) training on preference data. 3. PPO fine-tuning using the reward model's feedback.

3.1 Datasets

Three primary datasets were utilized in this project, each serving a distinct purpose in the RLHF pipeline.

- **Supervised Fine-Tuning (SFT) Data:** I went with the `yahma/alpaca-cleaned` dataset [Vanroy, 2023, vinhtran2611, 2023]. It's basically a cleaned-up version of the original Alpaca dataset, which has about 52,000 instruction-response examples generated by Open AI's older `text-davinci-003` model. Each example has an instruction, sometimes an input, and an output. The "cleaned" part means someone already fixed the hallucinations and weird errors in the original, so it's pretty solid for training. I only used 1% of it though—Colab would've cried if I tried the whole thing. I just formatted each example as `"Instruction:{instruction}\nAnswer:{output}"` and fed that to the model.

- **Reward Modeling (RM) Data:** This one was tricky. Ideally, you'd want a big dataset like **Anthropic's HH-RLHF** [Bai et al., 2022], which has tons of conversations where humans picked the better response. But honestly, that dataset is huge and my poor Colab GPU would've melted. So I did the next best thing: I made my own tiny dataset by hand. Just two examples, each with a prompt, a good response (chosen), and a bad one (rejected). It's nowhere near enough for a real reward model, but it worked as a proof-of-concept to test if the training pipeline itself was set up right.

- **RL Fine-Tuning Data:** I needed something to prompt the model during PPO, so I grabbed the **Stanford Sentiment Treebank (SST-2)** dataset [Moura, 2025, Data, 2025]. It's a standard sentiment analysis dataset—just sentences from movie reviews with

positive/negative labels. I didn't care about the labels here; I just used the sentences themselves as prompts. I filtered out anything shorter than 8 words so the model had something interesting to work with, which left me with about 31k training prompts and 807 for validation. Using SST-2 was a bit of a hack—it's not an instruction dataset at all—but I wanted to see if the model could generate fluent, relevant continuations in a zero-shot setting without being told what to do.

3.2 Models and Tokenizer

Due to working with limited hardware that being just a Colab T4 GPU, I had to pick small models. I went with two so I could compare how model size affects RLHF [Li et al., 2021].

- **Base Model 1: GPT-2 (124 Million parameters).** This is the standard GPT-2 small model—12 layers, 12 attention heads, hidden size of 768. It was originally trained on OpenAI's WebText dataset. I used it as my "bigger" baseline (even though 124M is tiny by today's standards).
- **Base Model 2: DistilGPT2 (82 Million parameters).** This is a smaller version of GPT-2, created using knowledge distillation [Face, 2023a,b]. It only has 6 layers (half of GPT-2), so it's about 33% smaller and roughly twice as fast. But compression comes with trade-offs. According to its documentation, on the WikiText-103 benchmark, GPT-2 gets a perplexity of **16.3** while DistilGPT2 gets **21.1** [Face, 2023a,b]. That higher perplexity basically means it's more "confused"—it makes more mistakes predicting the next token. I figured this would probably hurt it in more complex tasks like RLHF.
- **Tokenizer:** I just used the default tokenizer for each model. One thing I had to do was set the end-of-sequence token (<|endoftext|>) as the padding token so I could batch sequences of different lengths.

Table 1 below shows the main differences between the two models.

Table 1: Architectural Comparison of GPT-2 and DistilGPT2

Model	Parameters	Layers	Speed (Inference)	WikiText-103 PPL (Fine-tuned)
GPT-2 (small)	124M	12	1x (Baseline)	16.3
DistilGPT2	82M	6	∼2x faster	21.1

Source: Hugging Face Model Documentation [Face, 2023a]

3.3 Stage 1: Supervised Fine-Tuning (SFT)

I ran SFT on both GPT-2 and DistilGPT2. The whole point was to get them used to the instruction-response format from the Alpaca dataset.

- **Process:** I took that 1% subset of Alpaca, formatted it, and tokenized it. Then I just used Hugging Face's `Trainer` with `DataCollatorForLanguageModeling` and let it run for one epoch. Nothing fancy—just kept an eye on the training loss to make sure something was actually happening.

- **Outcome:** Both models picked up the style well enough. Like I expected, DistilGPT2's loss was higher and all over the place compared to GPT-2—just confirms that it's got less capacity to work with. I saved both models locally so I could use them later.

3.4 Stage 2: Reward Modeling (RM)

This is where I built a model to score responses based on what humans prefer. Quick note: after seeing how rough DistilGPT2's SFT outputs were, I decided to only use GPT-2 for this stage and the PPO part. DistilGPT2 just wasn't cutting it.

- **Model Architecture:** I made a custom reward model I called `GPT2RewardHead`. Basically, I took the base GPT-2 and slapped a randomly initialized linear layer on top of the last hidden state. That layer spits out a single number—the reward score. I added a sigmoid at the end to squish it between 0 and 1, which I treated as the probability that a response was the "chosen" one.
- **Data Preparation:** My tiny hand-made preference dataset got tokenized. For each example, I just concatenated the prompt with the chosen response and the prompt with the rejected response separately.
- **Training:** I trained it with Binary Cross-Entropy loss (BCELoss). The idea was simple: for the same prompt, push the score for the chosen response up and the rejected one down.

3.5 Stage 3: RL Fine-Tuning with PPO

The last stage was the most involved. I used PPO to fine-tune the SFT models further, using the reward model as a guide. I wrote my own PPO implementation instead of using a library—wanted to really understand what's going on under the hood. I ran this for both GPT-2 and DistilGPT2 to see how they'd compare.

- **Model with Value Head:** I wrapped the SFT model in a new class called `ModelForCausalLMWithValueHead`. This just adds a "value head"—another linear layer, that PPO needs to estimate how good a state is.
- **Generation:** For every prompt from the filtered SST-2 training set, the current policy model (the one being trained) generated a response.
- **Reward Calculation:** I passed the full prompt+response sequence to the reward model (which I kept frozen). The reward at the very end was: reward $= -\beta \cdot kl + \text{score}_{\text{at end}}$ The kl term is the per-token KL divergence between the new policy and the original SFT model—it keeps the model from drifting too far.

$$\text{reward} = \text{score} - \beta \cdot \log\left(\frac{\pi_\theta^{RL}}{\pi^{SFT}}\right) \tag{1}$$

- **PPO Update:** The policy (LLM) and value head are updated using mini-batches. The PPO loss function includes a clipped surrogate objective to ensure stable policy updates.

$$\mathcal{L}^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{2}$$

4 Evaluation and Experimental Results

This section presents the results of the RLHF pipeline applied to both GPT-2 and DistilGPT2, directly addressing the research question: **Does model capacity significantly impact the effectiveness of Reinforcement Learning from Human Feedback (RLHF) in small, open-source language models?**

According to my research, model capacity definitely plays a role in the success of RLHF in smaller LLMs. As we saw from the results, DistilGPT2 which is smaller performed poorly after RLHF pipeline was implemented whereas GPT-2 performed much better. Figuring out whether RLHF actually worked isn't straightforward—ideally you'd want humans to judge the outputs. Since I couldn't do that at scale, I relied on the reward model's scores (scaled to a range of $[-1, 1]$) and qualitative comparisons of generated outputs.

4.1 Supervised Fine-Tuning (SFT) Results

Both models fine-tuned successfully, though their generated responses revealed clear differences in quality and informativeness.

- **Prompt:** "Instruction: Explain what is gen AI.\nAnswer:"
- **DistilGPT2 SFT Output:** "Gen AI is a system that learns and improves."
- **GPT-2 SFT Output:** "Gen AI is a natural language system that is developed to create intelligent and complex AI systems that are able to make decisions in a variety of fields and industries. It is used to analyze data and analyze the underlying information and learn from it."

Training dynamics revealed notable differences between the two models:

- **DistilGPT2:** Training loss decreased consistently from 3.07 at step 10 to 1.73 at step 250, achieving a final training loss of 1.94.
- **GPT-2:** Training plateaued earlier, with loss values stabilizing around 4.3-4.6 and showing no further improvement, indicating potential challenges in learning from the instruction-following dataset.

The DistilGPT2 response, while grammatically correct, is excessively brief and lacks substantive explanation. GPT-2, despite its higher plateaued loss, makes a more elaborate attempt to explain the concept. This suggests that loss alone does not fully capture generation quality, and that GPT-2's larger capacity (124M parameters, 12 layers) allows it to produce more verbose outputs even with higher loss values, while DistilGPT2's compressed architecture (82M parameters, 6 layers) limits its expressive capability [Li et al., 2021].

4.2 PPO Training Dynamics and Reward Analysis

For the PPO phase, I tracked the average reward score on a validation set of 807 prompts (filtered from SST-2) both before and after training. The reward model outputs were transformed using score $= 2 \times (\text{sigmoid output} - 0.5)$, scaling them to a range of $[-1, 1]$.

- **GPT-2:** Average reward increased from 0.0648 to 0.1185, an improvement of approximately **83%**.
- **DistilGPT2:** PPO training completed over 195 steps, but loss values were not properly tracked, and the final model outputs demonstrated significant degradation in quality.

The substantial increase for GPT-2 demonstrates that the PPO optimization was effective. The model successfully adapted its policy to generate responses that the reward model judged more favorably, indicating successful alignment with the proxy human preference signal.

4.3 Qualitative Analysis of Model Outputs

To understand what these numerical differences represent in practice, Table 2 shows responses from both models before and after PPO fine-tuning.

Model	Stage	Output
DistilGPT2	SFT	"Gen AI is a system that learns and improves."
	PPO	"Gen AI is the perfect tool... You're not a 'doomsday' moment... A federal judge... stepping down from the Obama administration's ban..."
GPT-2	SFT	"for young women who find themselves of color for many during the years"
	PPO	"is surprisingly watchable, though it is not for everyone"

Table 2: Comparative model outputs before and after PPO fine-tuning. GPT-2 shows clear improvement in coherence and relevance, while DistilGPT2 degrades into incoherence.

The difference is striking and directly answers our research question:

- **GPT-2 (124M parameters):** The SFT output is largely incoherent—it starts generating a phrase but fails to form a complete or meaningful sentence. In contrast, the PPO-tuned model produces a fluent, coherent, and contextually relevant response. It demonstrates an understanding of the prompt ("an artsploitation movie") and offers a nuanced critical opinion. This qualitative leap, paired with the quantitative 83% increase in reward score, confirms that GPT-2 possessed sufficient representational capacity to learn from the reward signal and substantially improve its generation quality.
- **DistilGPT2 (82M parameters):** The SFT output, while brief, is at least on-topic. However, after PPO fine-tuning, the output becomes completely incoherent and off-topic—discussing White House politics, federal judges, and immigration bans when asked about generative AI. This degradation demonstrates that DistilGPT2 lacked the capacity to benefit from RLHF. Its compressed architecture (6 layers vs. GPT-2's 12 layers) and higher perplexity

(21.1 vs. 16.3 on WikiText-103) prevented it from effectively learning from the reward signal.

4.4 Answer to Research Question

The evidence overwhelmingly supports an affirmative answer to the research question: **Yes, model capacity significantly impacts the effectiveness of RLHF in small, open-source language models.**

GPT-2 (124M parameters) demonstrated clear improvement through RLHF, with an 83% increase in average reward score and qualitatively better, more coherent outputs. DistilGPT2 (82M parameters), despite successful SFT, failed to benefit from PPO fine-tuning, producing degraded, incoherent outputs. This confirms that a model must have sufficient representational capacity to effectively absorb and be refined by preference-based feedback. The compression that makes DistilGPT2 faster and smaller (33% fewer parameters, half the layers) comes at the cost of the flexibility needed for complex alignment tasks like RLHF.

5 Limitations and Future Work

The pipeline worked—I got it running, saw improvements, learned a ton. But there were definitely some major limitations I couldn't get around.

- **Model Size:** I was stuck with small models because of hardware. GPT-2 (124M) and DistilGPT2 (82M) are tiny. And DistilGPT2's poor performance? That's exactly what the compression papers talk about—distillation loses information, you get higher perplexity, and complex tasks like RLHF just suffer [Li et al., 2021, Face, 2023a]. It wasn't that DistilGPT2 was badly trained—it just didn't have the capacity to begin with.
- **Data Quality:** My reward model was trained on a whopping... two examples. Yeah. You need thousands of high-quality preferences (like from **Anthropic HH-RLHF** [Bai et al., 2022]) to train a decent reward model. Mine barely counts as a proof of concept. It probably learned a very weird, narrow idea of what "good" looks like.
- **Reward Hacking:** This is always a risk with RLHF. The model might find ways to game the reward model—generate stuff that scores high but is actually nonsense. I didn't see obvious hacking in my samples, but that 83% reward jump for GPT-2? Could be a early warning sign.
- **Lack of Human Evaluation:** I mostly relied on the reward score and a handful of examples. Real evaluation needs actual humans doing blind comparisons, like in the InstructGPT paper [Ouyang et al., 2022]. I just didn't have the setup for that.

5.1 Future Work

If I had more time/compute/resources, here's what I'd try:

1. **Bigger Models:** Something like Llama 2 7B or Mistral 7B. Enough capacity to actually see what RLHF can do.

2. **Real Preference Data:** Grab a proper dataset like **Anthropic's HH-RLHF** [Bai et al., 2022] instead of hacking together two examples. Train a reward model that actually knows what it's doing.

3. **Use Existing Libraries:** I wrote my own PPO code to learn, which was great for understanding, but libraries like TRL or OpenRLHF are way more optimized and less bug-prone. I was running into version issues with my TRL library so i decided not to use it extensively.

4. **Exploring DPO:** I'd love to compare PPO against DPO [Rafailov et al., 2023]—it's simpler, skips the reward model entirely, and might work better with limited data.

References

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

DTU Data. Bnns for sst2 as developed in on local posterior structure in deep ensembles, 2025. URL `https://data.dtu.dk/articles/dataset/BNNs_for_SST2_as_developed_in_On_Local_Posterior_Structure_in_Deep_Ensembles/25880989`. Accessed: [March 14, 2026].

Hugging Face. Distilgpt2 model card, 2023a. URL `https://huggingface.co/distilgpt2`. Accessed: [March 13, 2026].

Hugging Face. Transformers distillation examples readme, 2023b. URL `https://github.com/huggingface/transformers/tree/main/examples/research_projects/distillation`. Accessed: [March 13, 2026].

Tao Li, Yassir El Mesbahi, Ivan Kobyzev, Ahmad Rashid, Ahmad Mahmud, Nandakishore Anchuri, and Mehdi Rezagholizadeh. A short study on compressing decoder-based language models. *arXiv preprint arXiv:2110.08460*, 2021.

Pedro Henrique Batista de Moura. Entropy-guided transformers for sentiment prediction. In *Northrop Grumman Engineering & Science Student Design Showcase*, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

Bram Vanroy. Alpaca cleaned dutch, 2023. URL https://huggingface.co/datasets/BramVanroy/alpaca-cleaned-dutch. Accessed: [March 13, 2026].

vinhtran2611. yahma_alpaca-cleaned_formated, 2023. URL https://huggingface.co/datasets/vinhtran2611/yahma_alpaca-cleaned_formated. Accessed: [March 13, 2026].